Day 67: RNN vs LSTM vs GRU — a Real Comparison on Week 9's Tagging Task

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 5 | Complete study guide

Watch: Stanford CS224N (Winter 2019), Lecture 7: Vanishing Gradients, Fancy RNNs (Stanford Online) | MIT 6.S191 (2020): Recurrent Neural Networks (Alexander Amini)

Script: `day67_rnn_lstm_gru_compared.py` · every number in this guide is copied by program from `day67_run_output.txt`, the real output of that script.

Days 63–66 showed that gated cells carry information much further than the plain RNN, on a task built to need long memory. Today asks whether that matters on a real task: Day 62's part-of-speech tagger, with only the recurrent cell swapped. It matters a little. The LSTM tagger reached 0.8377 test accuracy, the GRU 0.8336 and the RNN 0.8327 (mean of three seeds each). The look-back probe shows why the gap is small: all three taggers rely almost entirely on the last few words, where the plain RNN's memory is already good enough.

0. Where today fits

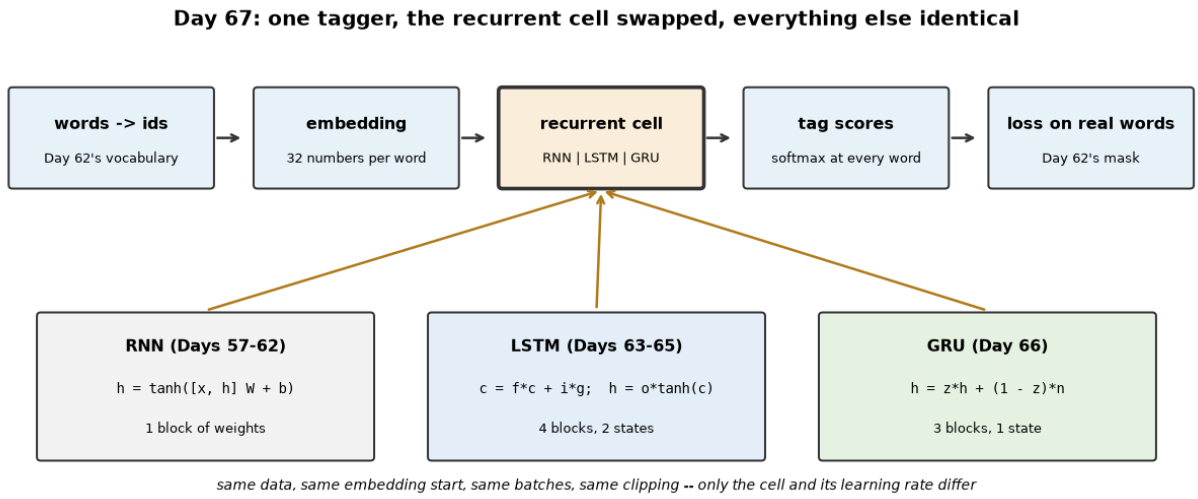
DEFINITION — start here, no prior knowledge assumed

A **controlled comparison** changes one thing and keeps everything else identical, so any difference in the result can only come from that one thing. Here the one thing is the **recurrent cell**: the plain RNN (Days 57–62), the LSTM (Days 63–65) or the GRU (Day 66), inside Day 62's tagger.

REAL-WORLD SCENARIO

Testing three engines in the same car, on the same road, with the same driver. A race track built for top speed (Day 59's sign task) showed one engine is far faster. Today is the daily commute through town: does the faster engine still get you to work sooner?

SYSTEM DIAGRAM



Day 62's tagger with a slot for the cell. Data, embedding start, batches and clipping are identical; only the cell and its learning rate differ.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. One tagger, three cells

DEFINITION — start here, no prior knowledge assumed

The tagger turns each word into an id, looks up a vector of 32 numbers for it (the **embedding**), runs a recurrent cell over the sentence and scores the 17 tags at every word. A **Tagger** class with a `cell` setting builds any of the three versions; the forward and backward passes are Days 62, 64–65 and 66's code, unchanged.

REAL-WORLD SCENARIO

One kitchen, one menu, three different ovens slotted into the same space. The recipes, ingredients and plates stay the same.

MATHEMATICS

$$\textit{cell weights: RNN } (D + H + 1)\,H,\quad \textit{GRU } 3\,(D + H + 1)\,H,\quad \textit{LSTM } 4\,(D + H + 1)\,H \quad (D = H = 32)$$

$$\textit{embedding: vocabulary} \times D = 2082 \times 32;\quad \textit{output layer: } (H + 1) \times 17$$

SPECIFICATION

Shared	Vocabulary 2,082 words, 17 tags, embedding 32, hidden 32, Day 62's data and splits.
Same start	The embedding table is drawn first from the seed, so all three cells start from identical word vectors; the RNN is initialised exactly as on Day 62.

Why this test

Before comparing accuracy, it helps to know how much of each network the cell actually is.

IMPLEMENTATION

```
def run_cell(self, X):
    h0 = np.zeros((X.shape[0], self.hidden_dim))
    if self.cell == "rnn":
        return rnn_forward(X, h0, self.Wxh, self.Whh, self.bh), h0
    if self.cell == "lstm":
        Hs, _, self.cache = lstm_forward(X, h0, h0, self.W, self.b)
        return Hs, h0
```

ACTUAL OUTPUT

```
cell    embedding  cell weights  output layer    total  cell share
RNN         66624          2080           561    69265        3.0%
LSTM        66624          8320           561    75505       11.0%
GRU         66624          6240           561    73425        8.5%
Actual output from this lesson's run.
```

What it means

The LSTM's cell has four times the RNN's weights (8320 against 2080), but the embedding table (66624 numbers) dominates every version: the cell is only 3.0% of the RNN tagger and 11.0% of the LSTM tagger. Most of what any of these taggers learns is word vectors.

WHY CHOOSE THIS?

One class, one switch: the comparison cannot be spoiled by an accidental difference elsewhere.

WHY NOT THIS? (TRADEOFFS)

Each cell keeps its own initialisation and gate bias, which are part of what is being compared.

2. A gradient check for all three cells

DEFINITION — start here, no prior knowledge assumed

A **gradient check** nudges every weight up and down by a tiny amount, measures how the loss changes, and compares that slope with what the backward pass computed. Day 62's exact check, run once per cell.

REAL-WORLD SCENARIO

Weighing each oven's output on the same calibrated scale before the cook-off, so a broken scale cannot decide the winner.

SPECIFICATION

Data	12 words, 3 tags, embedding 4, hidden 5; four sentences of lengths 5, 2, 4, 1, one word repeated.
Pass rule	|numeric – analytic| ≤ 1e-10 + 1e-6 · |numeric| for every entry.

Why this test

The cells' backward passes were checked on Days 62, 65 and 66, but inside different models. This checks each one through the exact tagger that is trained today.

IMPLEMENTATION

```
for cell in CELLS:
    net = Tagger(12, 3, cell=cell, embed_dim=4, hidden_dim=5, clip=None, seed=1)
    net.forward(ids)
    analytic = net.gradients(tags, mask)
    ok_all = True
    for name in net.params:
        numeric = numeric_gradient(net, name, ids, tags, mask)
        diff = np.abs(numeric - analytic[name])
        rel = diff / (np.abs(numeric) + np.abs(analytic[name]) + 1e-12)
        ok = bool(np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric)))
        ok_all = ok_all and ok
        print(f"{NAMES[cell]:<6}{name:>10}{numeric.size:>9}{rel.max():>20.2e}{diff.max():>18.2e}{s
            tr(ok):>7}")
    verdict[cell] = ok_all and bool(np.all(analytic["E"][PAD] == 0))
```

ACTUAL OUTPUT

```
cell   parameter  entries  max relative error  max |difference|   pass
RNN            E       48            1.92e-07          2.45e-10   True
RNN          Wxh       20            5.30e-08          5.75e-11   True
RNN          Whh       25            3.12e-06          4.01e-11   True
RNN           bh        5            5.49e-09          1.83e-10   True
RNN          Why       15            1.37e-08          7.76e-11   True
RNN           by        3            7.30e-09          1.63e-10   True
LSTM           E       48            6.71e-08          3.90e-11   True
LSTM           W      180            3.42e-05          2.13e-11   True
LSTM           b       20            1.01e-06          1.02e-10   True
LSTM         Why       15            1.05e-07          2.32e-11   True
LSTM          by        3            2.86e-09          1.20e-11   True
GRU            E       48            2.14e-07          3.71e-11   True
GRU            W      135            1.62e-04          2.30e-11   True
GRU            b       15            8.86e-07          1.00e-10   True
GRU          Why       15            8.47e-09          1.71e-11   True
GRU           by        3            4.52e-09          6.29e-12   True
every entry passes, and <pad>'s embedding gradient is exactly zero: RNN True, LSTM True, GRU True
Actual output from this lesson's run.
```

What it means

Every entry of every parameter passes for all three cells. The largest relative errors (1.62e-04 for the GRU's W) sit on entries whose gradient is itself almost zero; the absolute difference there is 2.30e-11, which is why Day 62's rule combines an absolute and a relative allowance.

WHY CHOOSE THIS?

Rules out a bug as the explanation for any difference found below.

WHY NOT THIS? (TRADEOFFS)

A passing check says the gradients are right, not that the training settings are good.

3. Fair settings: each cell's own learning rate, chosen on validation

DEFINITION — start here, no prior knowledge assumed

A **learning rate** sets how big each training step is. Different cells can prefer different ones. **Validation** data is held out from training and used only to choose settings; the test data stays untouched until the end, so it gives an honest final score.

REAL-WORLD SCENARIO

Each engine gets tuned by its own mechanic before the race, using practice laps, not the race itself.

SPECIFICATION

Grid	Learning rates 0.5, 1, 2, 4 for every cell; gate bias 0 or 1 for the LSTM (forget gate) and the GRU (update gate). Seed 0, 25 epochs, batch 32, clip 1.0.
Rule	Keep the setting with the best validation accuracy at its best epoch.

Why this test

With one shared learning rate, the comparison would partly measure which cell that rate happens to suit.

IMPLEMENTATION

```
lrs = [0.5, 1.0, 2.0, 4.0]
grid = {"rnn": [(lr, 0.0) for lr in lrs],
        "lstm": [(lr, gb) for gb in [0.0, 1.0] for lr in lrs],
        "gru": [(lr, gb) for gb in [0.0, 1.0] for lr in lrs]}
```

```
print(f"\n{'cell':<6}{'lr':>5}{'gate bias':>11}{'best val':>10}{'at epoch':>10}{'val at
    end':>12}{'train at end':>14}")
chosen = {}
for cell in CELLS:
    best = (-1.0, None)
    for lr, gb in grid[cell]:
        _, be, h = fit_cached(data, V, K, seed=0, cell=cell, lr=lr, gate_bias=gb)
        print(f"{NAMES[cell]:<6}{lr:>5}{gb if cell != 'rnn' else
            '-':>11}{h['val'][be]:>10.4f}{be:>10}"
              f"{h['val'][-1]:>12.4f}{h['train'][-1]:>14.4f}")
        if h["val"][be] > best[0]:
            best = (h["val"][be], (lr, gb))
    chosen[cell] = best[1]
```

ACTUAL OUTPUT

```
cell     lr  gate bias  best val  at epoch  val at end  train at end
RNN     0.5          -    0.8170        21      0.8081        0.9122
RNN     1.0          -    0.8254        21      0.8092        0.9277
RNN     2.0          -    0.8232         9      0.8156        0.9440
RNN     4.0          -    0.7478        20      0.7180        0.8139
LSTM    0.5        0.0    0.8081        24      0.8081        0.8772
LSTM    1.0        0.0    0.8276        21      0.8197        0.9092
LSTM    2.0        0.0    0.8331        21      0.8224        0.9339
LSTM    4.0        0.0    0.8300         9      0.8137        0.9563
LSTM    0.5        1.0    0.8078        24      0.8078        0.8761
LSTM    1.0        1.0    0.8276        21      0.8274        0.9101
LSTM    2.0        1.0    0.8327        18      0.8289        0.9313
LSTM    4.0        1.0    0.8333        13      0.8143        0.9530
GRU     0.5        0.0    0.8169        24      0.8169        0.8918
GRU     1.0        0.0    0.8331        21      0.8248        0.9163
GRU     2.0        0.0    0.8288        18      0.8230        0.9390
GRU     4.0        0.0    0.8333         9      0.8018        0.9645
GRU     0.5        1.0    0.8093        24      0.8093        0.8822
GRU     1.0        1.0    0.8274        21      0.8236        0.9093
GRU     2.0        1.0    0.8285        18      0.8218        0.9343
GRU     4.0        1.0    0.8286         9      0.8129        0.9544
chosen on validation: RNN lr 1.0, LSTM lr 4.0 gate bias 1.0, GRU lr 4.0 gate bias 0.0
```

What it means

The choices were: RNN lr 1.0; LSTM lr 4.0 with forget bias 1.0; GRU lr 4.0 with keep bias 0.0. The winners are close to their runners-up: the LSTM's 0.8333 against 0.8331 at lr 2.0 and bias 0. The validation set has 6,634 words, so one standard error there is about 0.005. Many settings in this grid are equally good, and the exact choice is partly luck.

The gated cells start more slowly (they need several epochs before they overtake the RNN, see the curves in Section 4), so they preferred larger learning rates within the same 25-epoch budget.

WHY CHOOSE THIS?

Every cell is compared at a setting chosen for it, using data the final score never sees.

WHY NOT THIS? (TRADEOFFS)

A grid of one seed cannot separate settings closer than the noise; a wider search could shift each cell a little.

4. The comparison: three seeds per cell

DEFINITION — start here, no prior knowledge assumed

Each cell is trained with three different random seeds at its chosen setting. For each run, the test accuracy is read at the epoch with the best validation accuracy. Several seeds show how much of a difference is just the luck of the starting weights.

REAL-WORLD SCENARIO

Running the commute three times with each engine, on different days, before deciding which is faster.

MATHEMATICS

standard error of an accuracy p on n words: $\sqrt{p\,(1-p)\,/\,n}$ *(here n = 18,460 test words)*

SPECIFICATION

Runs	3 cells × seeds 0, 1, 2, at each cell's chosen setting; 25 epochs.
Baseline	Day 62's most-frequent-tag: every word gets its most common training tag, no memory at all.
Check	The RNN at Day 62's own setting must reproduce Day 62's 0.8338.

Why this test

The headline question: on real data, is one cell better?

IMPLEMENTATION

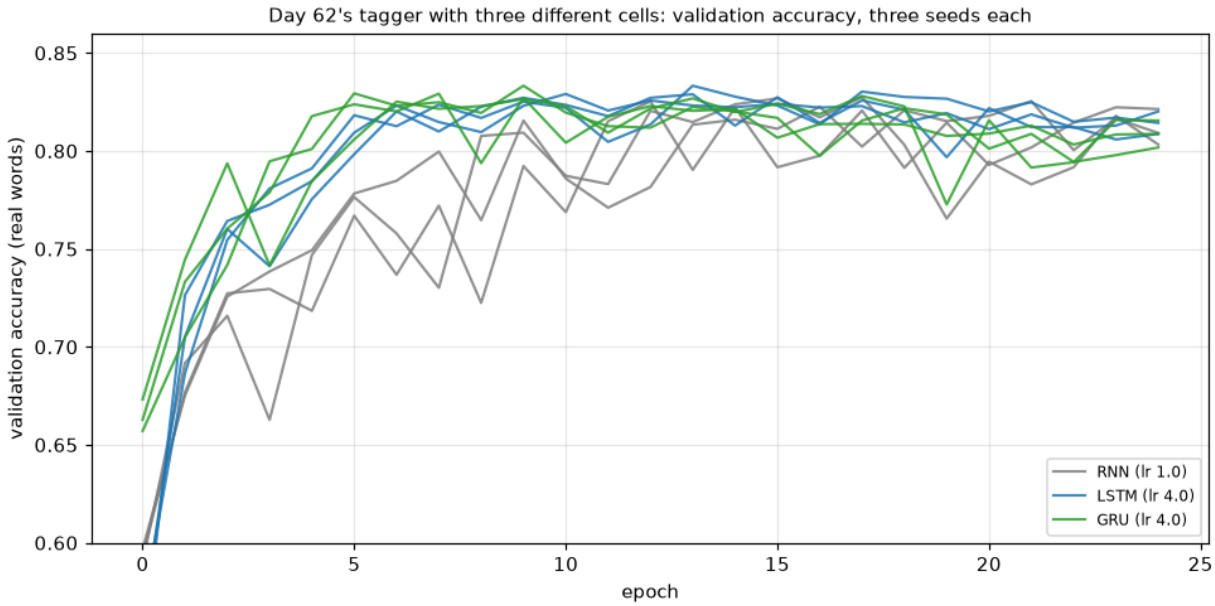
```
tests[cell] = []
for seed in SEEDS:
    _, be, h = fit_cached(data, V, K, seed=seed, cell=cell, lr=lr, gate_bias=gb)
    tests[cell].append(h["test"][be])
    print(f"{NAMES[cell]:<6}{seed:>5}{be:>12}{h['val'][be]:>11.4f}{h['test'][be]:>12.4f}{h['train'
        ][-1]:>14.4f}")
```

ACTUAL OUTPUT

```
cell   seed  best epoch  val there  test there  train at end
RNN       0          21     0.8254      0.8338        0.9277
RNN       1          12     0.8253      0.8334        0.9324
RNN       2          17     0.8274      0.8308        0.9277
LSTM      0          13     0.8333      0.8395        0.9530
LSTM      1          10     0.8291      0.8370        0.9596
LSTM      2          13     0.8289      0.8366        0.9558
GRU       0           9     0.8333      0.8378        0.9645
GRU       1           7     0.8292      0.8311        0.9691
GRU       2          17     0.8280      0.8320        0.9676
cell    mean test     min     max  vs most-frequent-tag
RNN        0.8327  0.8308  0.8338               +0.0310
LSTM       0.8377  0.8366  0.8395               +0.0361
GRU        0.8336  0.8311  0.8378               +0.0320
most-frequent-tag baseline on test: 0.8016; test words 18460, one standard error about 0.0027
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
consistency check: the RNN at Day 62's setting (lr 1.0, seed 0) reaches test 0.8338 at its best
    validation epoch (Day 62 reported 0.8338)
Actual output from this lesson's run.
```



Validation accuracy per epoch, every seed. The gated cells (blue, green) climb faster after the first epochs; all three level off around the same height and then drift down as they overfit.

What it means

The LSTM is a little better, consistently. Its mean 0.8377 is +0.0361 above the baseline against the RNN's +0.0310, and its worst seed (0.8366) beats the RNN's best (0.8338). The gap, 0.005, is about twice the standard error of 0.0027. The GRU (0.8336) is level with the RNN: its seeds range from 0.8311 to 0.8378.

The setup is trustworthy: the RNN reproduces Day 62's number exactly (0.8338 against 0.8338). And every tagger overfits: training accuracy ends at 0.93–0.97 while test stays near 0.83–0.84. The limit is the small training set (25,148 words) and the 20% unknown words, not the cell.

WHY CHOOSE THIS?

On this task the LSTM bought about half a percentage point, reliably across seeds.

WHY NOT THIS? (TRADEOFFS)

Half a point, for 3.4 times the training time (Section 8). The GRU bought nothing measurable here.

5. Where the cells differ

DEFINITION — start here, no prior knowledge assumed

Day 62's three word groups: **unambiguous** (a known word that had one tag in training), **ambiguous** (a known word that had several, like “run”), **unknown** (not in the vocabulary). Plus the word's position in its sentence, and how often two cells disagree.

REAL-WORLD SCENARIO

Checking which streets the faster engine actually saved time on: highways, side streets, or the unfamiliar roads.

SPECIFICATION

Groups	37.7% / 42.2% / 20.2% of the test words; positions 1st, 2nd–3rd, 4th–10th, 11th–20th, 21st on.
Disagreement	Seed 0 of each pair of cells: share of words tagged differently, and who was right then.

Why this test

A small overall gap can hide a large gap in one group, or many small gaps that cancel.

IMPLEMENTATION

```
pos = np.tile(np.arange(ids_te.shape[1]), (len(ids_te), 1))
spans = [(0, 1), (1, 3), (3, 10), (10, 20), (20, 100)]
span_names = ["1st word", "2nd-3rd", "4th-10th", "11th-20th", "21st on"]
for (a, b), name in zip(spans, span_names):
    groups[name] = mask_te & (pos >= a) & (pos < b)
```

ACTUAL OUTPUT

```
by word group:
              all words unambiguous   ambiguous     unknown
share            100.0%       37.7%       42.2%       20.2%
most-freq        0.8016      0.9814      0.8474      0.3705
RNN              0.8327      0.9693      0.8740      0.4914
LSTM             0.8377      0.9732      0.8749      0.5072
GRU              0.8336      0.9744      0.8666      0.5018
by position in the sentence:
               1st word     2nd-3rd    4th-10th   11th-20th     21st on
share              8.9%       15.8%       38.1%       24.8%       12.4%
most-freq        0.7656      0.7878      0.7993      0.8219      0.8119
RNN              0.8231      0.8381      0.8307      0.8365      0.8310
LSTM             0.8199      0.8364      0.8386      0.8435      0.8379
GRU              0.8233      0.8389      0.8326      0.8347      0.8354
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
how often two cells (seed 0 each) give a word DIFFERENT tags, and who is right then:
pair          disagree  first right  second right  neither
RNN-LSTM         9.93%        35.7%         41.5%    22.8%
RNN-GRU         10.16%        36.8%         40.7%    22.5%
LSTM-GRU         8.08%        41.4%         39.2%    19.3%
Actual output from this lesson's run.
```

What it means

The LSTM's gain comes mostly from **unknown words** (0.5072 against the RNN's 0.4914) and from words **deeper in the sentence** (11th–20th: 0.8435 against 0.8365). On the first word, where no memory is possible, it is not better (0.8199 against 0.8231).

The cells disagree on about one word in ten (9.93% for RNN and LSTM), and when they disagree each is right a similar share of the time (35.7% RNN, 41.5% LSTM). They make different mistakes, not one strictly fewer.

WHY CHOOSE THIS?

Shows the gain is where memory should help: unknown words and later positions.

WHY NOT THIS? (TRADEOFFS)

The groups are smaller than the whole test set, so their differences are noisier.

6. How far back does each tagger look?

DEFINITION — start here, no prior knowledge assumed

A **probe** measures what a trained network actually uses. Here: replace one word k positions before a target word with a random word, and measure how much the target's tag probabilities move. **Total-variation distance (TV)** is half the sum of the absolute differences between two distributions: 0 means identical, 1 means no overlap.

REAL-WORLD SCENARIO

To find out how far back a reader really remembers a story, change one sentence a page earlier and see whether their summary of the current paragraph changes.

MATHEMATICS

$$TV(p, q) = \tfrac{1}{2} \Sigma_k \, | \, p_k - q_k \, |$$

SPECIFICATION

Edits	For each test sentence and distance k (1, 2, 3, 4, 6, 8, 12): a random target word, the word k before it replaced by a random vocabulary word; 5 draws per sentence; the same edits for every cell.
Models	The three seeds of each cell from Section 4.

Why this test

Day 59 measured how far gradient *can* travel in an untrained network. This measures how far the trained taggers actually look on real sentences.

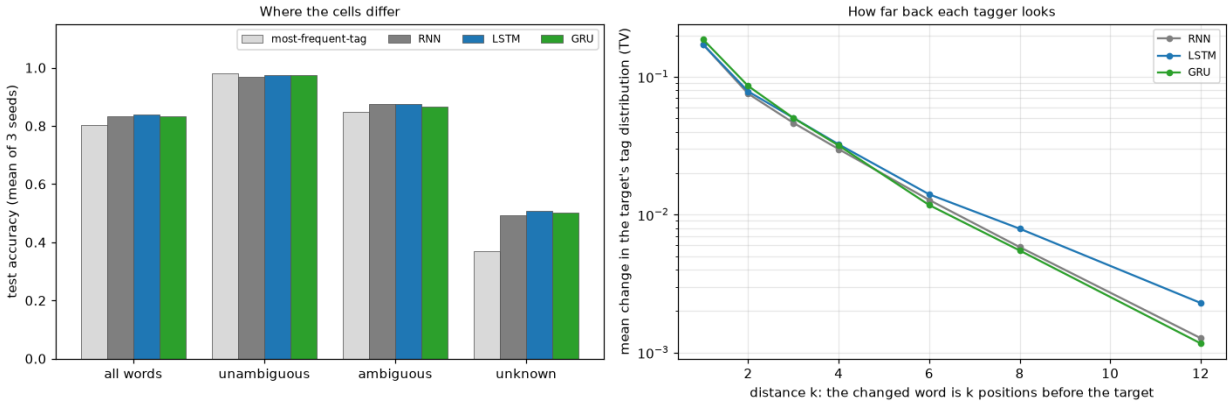
IMPLEMENTATION

```
rng = np.random.RandomState(100 + k)
rows = np.where(lens > k)[0]
draws = []
for _ in range(5):
    for r in rows:
        t = rng.randint(k, lens[r])
        draws.append((r, t, rng.randint(2, V)))
draws = np.array(draws)
counts.append(len(draws))
r_idx, t_idx, new_w = draws[:, 0], draws[:, 1], draws[:, 2]
changed = ids_te[r_idx].copy()
changed[np.arange(len(draws)), t_idx - k] = new_w
same = changed[np.arange(len(draws)), t_idx - k] == ids_te[r_idx, t_idx - k]
for cell in CELLS:
    flips, tvs = [], []
    for s in SEEDS:
        net = fit_cached(data, V, K, seed=s, cell=cell, lr=chosen[cell][0],
            gate_bias=chosen[cell][1])[0]
        p_orig = probs_at(net, ids_te[r_idx], t_idx)
        p_new = probs_at(net, changed, t_idx)
        keep = ~same
        flips.append((p_orig.argmax(1) != p_new.argmax(1))[keep].mean())
        tvs.append(0.5 * np.abs(p_orig - p_new).sum(axis=1)[keep].mean())
    results[cell]["flip"].append(np.mean(flips))
    results[cell]["tv"].append(np.mean(tvs))
```

ACTUAL OUTPUT

```
   k   draws   RNN changed  LSTM changed   GRU changed    RNN TV   LSTM TV    GRU TV
   1    7575        18.55%        18.13%        19.36%    0.1720    0.1721    0.1870
   2    7010         8.60%         8.23%         9.29%    0.0754    0.0783    0.0857
   3    6320         4.42%         5.37%         5.45%    0.0463    0.0503    0.0503
   4    5875         2.92%         3.42%         3.13%    0.0299    0.0323    0.0317
   6    5005         1.36%         1.44%         1.20%    0.0128    0.0140    0.0117
```

```
   8    4130          0.44%          1.06%          0.59%    0.0058    0.0079    0.0055
  12    2890          0.09%          0.16%          0.15%    0.0013    0.0023    0.0012
Actual output from this lesson's run.
```



Left: accuracy by word group. Right: how much changing the word k positions back moves the target's tag distribution (log scale).

What it means

All three taggers look mostly at the last few words. Changing the word just before the target changes the predicted tag in 18.55% (RNN), 18.13% (LSTM) and 19.36% (GRU) of cases; four words back it is about 3%; twelve words back it is 0.09%, 0.16% and 0.15%. The LSTM's curve falls a little more slowly beyond 6 words (TV 0.0079 at 8, against 0.0058 for the RNN), but by then the effect is tiny for every cell.

This is why the cell matters so little here: part-of-speech tags are decided mostly by the word itself and its near neighbours, a range where the plain RNN's memory already works. Day 59's long-memory advantage has little to act on.

WHY CHOOSE THIS?

Explains the size of the accuracy gap instead of just reporting it.

WHY NOT THIS? (TRADEOFFS)

It measures sensitivity to one random edit at a time; it does not show what each tagger would do with information it never needed.

7. A fairer size match

DEFINITION — start here, no prior knowledge assumed

At the same hidden size the LSTM's cell has four times the RNN's weights. A **size-matched** comparison gives the RNN a larger hidden state so both cells have about the same number of weights.

REAL-WORLD SCENARIO

Giving the smaller engine a turbocharger so both have the same horsepower, before comparing them again.

SPECIFICATION

Models	RNN hidden 76 (8,284 cell weights) and GRU hidden 38 (8,094) against the LSTM at 32 (8,320); each at its cell's chosen learning rate; seeds 0, 1, 2.

Why this test

If the LSTM only wins because it is bigger, a bigger RNN should catch up.

IMPLEMENTATION

```
for cell, hidden in [("rnn", 32), ("rnn", 76), ("gru", 38), ("lstm", 32)]:
    lr, gb = chosen[cell]
    accs = []
    for s in SEEDS:
        if hidden == 32:
            _, be, h = fit_cached(data, V, K, seed=s, cell=cell, lr=lr, gate_bias=gb)
        else:
            _, be, h = fit_cached(data, V, K, seed=s, cell=cell, lr=lr, gate_bias=gb,
                hidden_dim=hidden)
        accs.append(h["test"][be])
```

ACTUAL OUTPUT

```
model             cell weights   seed 0   seed 1   seed 2     mean
RNN hidden 32             2080   0.8338   0.8334   0.8308   0.8327
RNN hidden 76             8284   0.8242   0.8246   0.8281   0.8256
GRU hidden 38             8094   0.8360   0.8379   0.8323   0.8354
LSTM hidden 32            8320   0.8395   0.8370   0.8366   0.8377
Actual output from this lesson's run.
```

What it means

The bigger RNN does **worse** (0.8256 against 0.8327 at hidden 32): with more weights it overfits the small training set more. The GRU at 38 (0.8354) moves a little towards the LSTM (0.8377). The LSTM's edge is not just its size.

WHY CHOOSE THIS?

Separates “better design” from “more weights”.

WHY NOT THIS? (TRADEOFFS)

The bigger RNN kept the learning rate chosen for hidden 32; a retuned rate might do somewhat better.

8. What each cell costs

DEFINITION — start here, no prior knowledge assumed

Training time per epoch, measured inside every run above, with the matrix library on one thread.

REAL-WORLD SCENARIO

The faster engine also uses more fuel. Is the time saved worth it?

IMPLEMENTATION

```
secs = np.concatenate([fit_cached(data, V, K, seed=s, cell=cell, lr=chosen[cell][0],
    gate_bias=chosen[cell][1])[2]["seconds"]
                       for s in SEEDS])
med[cell] = np.median(secs)
```

ACTUAL OUTPUT

```
cell    seconds / epoch   vs RNN
RNN                0.13     1.00
LSTM               0.45     3.40
GRU                0.36     2.71
Actual output from this lesson's run.
```

What it means

An LSTM epoch took 3.40 times as long as an RNN epoch, a GRU epoch 2.71 times. For half a percentage point on this task, that is a real price; on a task that needs long memory (Day 65) it buys the difference between learning and not learning.

WHY CHOOSE THIS?

Measured on the actual workload, not a single step.

WHY NOT THIS? (TRADEOFFS)

Timings depend on the machine and the sizes.

Interview preparation — technical questions

Q: How do you compare two architectures fairly?

A: Change only the architecture; tune each one's learning rate on validation data; report several seeds; read test accuracy once, at the epoch validation picked; check that a known result is reproduced.

Q: Your LSTM beats your RNN by 0.5 points. Is that real?

A: Compare it with the noise: here the standard error is about 0.27 points and every LSTM seed beat every RNN seed, so it is small but consistent. Also check a size-matched baseline.

Q: How would you measure how much context a trained sequence model uses?

A: Perturb an input at distance k from the prediction and measure how much the output distribution moves, for several k: an occlusion or perturbation probe.

Q: Why did a bigger RNN do worse?

A: Most of these taggers' weights are the embedding and the training set is small; more weights mostly meant more overfitting.

Q: What is total-variation distance?

A: Half the sum of absolute differences between two probability distributions; 0 for identical, 1 for disjoint.

Interview preparation — theoretical questions

Q: When does an LSTM or GRU clearly beat a plain RNN?

A: When the task needs information from far back (Day 59's sign task, long documents). When the needed context is short, as in part-of-speech tagging, the difference is small.

Q: LSTM or GRU?

A: No general winner. Here the LSTM was slightly better and the GRU cheaper; on Day 66's long-memory task the LSTM was also more reliable. Try both when it matters.

Q: Why tune the learning rate per model?

A: Models with different structures respond differently to step size; one shared rate can favour one of them.

Q: What limits these taggers, if not the cell?

A: Unknown words (20% of the test set), a small training set, and reading only left to right: no tagger has seen the words after the one it is tagging.

Key takeaway and what's next

> **KEY TAKEAWAY**
> On a real task with short-range structure, the cell matters only a little. Like for like, the LSTM tagger was about half a point better than the RNN, consistently across seeds and not just because it is bigger. The GRU was level with the RNN. The probe shows why: every tagger relies on the last few words, where the plain RNN's memory is already adequate. Long memory is worth paying for when the task needs it, and the way to find out is to measure.

- Controlled comparison: one change, each model's learning rate tuned on validation, several seeds.
- Test accuracy: RNN 0.8327, LSTM 0.8377, GRU 0.8336; baseline 0.8016; standard error 0.0027.
- The RNN reproduced Day 62 exactly: 0.8338.
- Gains are on unknown words and later positions; the cells disagree on about 10% of words.
- Changing the previous word moves the prediction in about 18% of cases; 12 words back, about 0.1%.
- An RNN with as many weights as the LSTM did worse (0.8256): overfitting, not capacity, limits it.
- LSTM epochs cost 3.4 times the RNN's.

Suggested watch order, if starting from zero

- 1. MIT 6.S191 (2020): Recurrent Neural Networks (Alexander Amini) — RNNs, their gradient problems and LSTMs, in one lecture.
- 2. Stanford CS224N (Winter 2019), Lecture 7: Vanishing Gradients, Fancy RNNs (Stanford Online) — LSTM and GRU side by side, and when gated cells help; also previews bidirectional and multi-layer RNNs (Day 68).

Day 68 preview

- **Stacked and bidirectional RNNs.** Every tagger today reads left to right, so it decides a word's tag before seeing the words after it. Day 62 measured that a right-to-left tagger is right on many words the left-to-right one gets wrong. Day 68 builds a tagger that reads both ways, and one with two layers.
- Today's exercise script, `day67_exercise.py`, isolates an LSTM with an output and a loss at every step, the piece Day 69's decoder is built from.